

2-O'lchamli Numpy Massivlari (2D Arrays)

Ushbu darslikda biz Numpy kutubxonasida ikki o'lchamli massivlar bilan ishlash asoslarini, ularning atributlarini va ular ustida bajariladigan matematik amallarni ko'rib chiqamiz.

1. Kirish va massiv yaratish

Ikki o'lchamli massivni (matritsani) yaratish uchun biz ichma-ich joylashgan ro'yxatlardan foydalanamiz.

```
import numpy as np

# Ichma-ich joylashgan ro'yxat yaratish
a = [[11, 12, 13], [21, 22, 23], [31, 32, 33]]

# Ro'yxatni Numpy massiviga o'tkazish
A = np.array(a)
print("Matritsa A:\n", A)
```

```
Matritsa A:
[[11 12 13]
 [21 22 23]
 [31 32 33]]
```

2. Massiv Atributlari

Numpy massivlarining asosiy xususiyatlarini quyidagi atributlar orqali aniqlash mumkin:

- **ndim** : O'lchamlar soni (rank). 2D massivlar uchun bu doim 2 ga teng.
- **shape** : Massivning shakli (qatorlar va ustunlar soni).
- **size** : Massivdagi jami elementlar soni.

```
# O'lchamlar sonini aniqlash
print("O'lchami (ndim):", A.ndim)

# Shaklini aniqlash (qatorlar, ustunlar)
print("Shakli (shape):", A.shape)

# Jami elementlar soni
print("Elementlar soni (size):", A.size)
```

O'lchami (ndim): 2
Shakli (shape): (3, 3)
Elementlar soni (size): 9

3. Indekslish va Kesish (Indexing & Slicing)

Elementlarga murojaat qilish uchun kvadrat qavslardan [qator, ustun] ko'rinishida foydalanamiz.

```
# Ikkinchi qator, uchinchi ustun elementini olish (indeks 0 dan boshlanadi)
# A[1][2] yoki A[1, 2] ko'rinishida yozish mumkin
print("Element A[1, 2]:", A[1, 2]) # Natija: 23

# Birinchi qatorning birinchi elementini olish
print("Element A[0, 0]:", A[0, 0]) # Natija: 11

# Slicing (Kesish): Birinchi qatorning dastlabki ikki elementini olish
print("Kesish A[0, 0:2]:", A[0, 0:2])

# Dastlabki ikki qatorning oxirgi ustunini olish
print("Kesish A[0:2, 2]:", A[0:2, 2])
```

Element A[1, 2]: 23
Element A[0, 0]: 11
Kesish A[0, 0:2]: [11 12]
Kesish A[0:2, 2]: [13 23]

4. Matematik Amallar

4.1. Massivlarni Qo'shish

Ikki massivni qo'shish elementma-element bajariladi. Ikkala massivning shakli (shape) bir xil bo'lishi kerak.

```
X = np.array([[1, 0], [0, 1]])
Y = np.array([[2, 1], [1, 2]])

Z = X + Y
print("X + Y natijasi:\n", Z)
```

X + Y natijasi:
[[3 1]
[1 3]]

4.2. Skalyar Ko'paytirish

Massivni biror songa ko'paytirganda, uning har bir elementi shu songa ko'paytiriladi.

```
Z = 2 * Y
print("2 * Y natijasi:\n", Z)
```

```
2 * Y natijasi:
[[4 2]
 [2 4]]
```

4.3. Elementma-element ko'paytirish (Hadamard Product)

`*` operatori orqali ikki massivning mos elementlari bir-biriga ko'paytiriladi.

```
Z = X * Y
print("Elementma-element ko'paytirish (X * Y):\n", Z)
```

```
Elementma-element ko'paytirish (X * Y):
[[2 0]
 [0 2]]
```

4.4. Matritsaviy ko'paytirish (Dot Product)

Haqiqiy matritsalar ko'paytmasini hisoblash uchun `np.dot()` funksiyasidan foydalaniladi. Bunda birinchi matritsaning ustunlar soni ikkinchi matritsaning qatorlar soniga teng bo'lishi shart.

```
A = np.array([[0, 1, 1], [1, 0, 1]])
B = np.array([[1, 1], [1, 1], [-1, 1]])

C = np.dot(A, B)
print("Matritsaviy ko'paytma (Dot Product):\n", C)
```

```
Matritsaviy ko'paytma (Dot Product):
[[0 2]
 [0 2]]
```